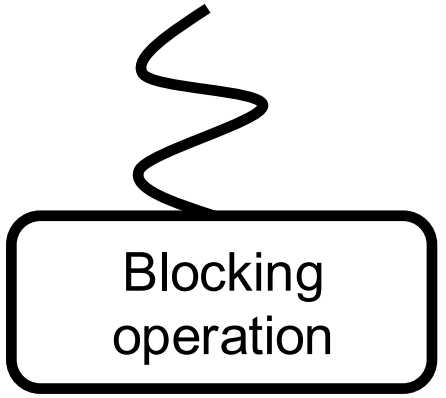
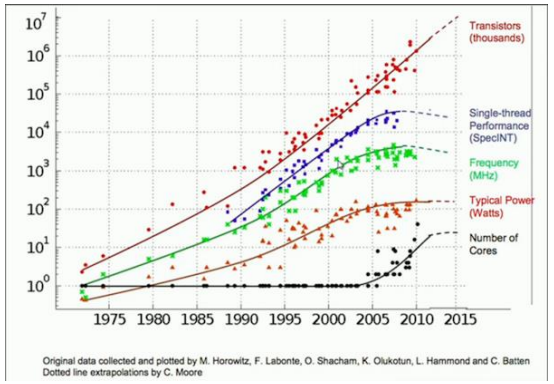
Concurrency



Review: Locks





Goal: Provide mutual exclusion (mutex)

Three common operations:

- Allocate and Initialize
  - `Pthread_mutex_t mylock = PTHREAD_MUTEX_INITIALIZER;`
- Acquire
  - Acquire exclusion access to lock;
  - Wait if lock is not available (some other process in critical section)
  - Spin or block (relinquish CPU) while waiting
  - `Pthread_mutex_lock(&mylock);`
- Release
  - Release exclusive access to lock; let another process enter critical section
  - `Pthread_mutex_unlock(&mylock);`

Review: LOCKS with Load+Store

Code uses a single **shared** lock variable

```
Boolean flag = false; // shared variable
Void lock(Boolean *flag) {
     while (*flag) /* wait */ ;
     *flag = true;
}
Void unlock(Boolean *flag) {
     *flag = false;
}
```



Use explicit hardware support!

Does this provide correct mutual exclusion?

Can you produce an example schedule that fails with 2 threads?

Review: Locks with Interrupts

Turn off interrupts for critical sections

- Prevent dispatcher from running another thread
- Code between interrupts executes atomically

```
void lock(lock_t *l) {
      disableInterrupts();
}

void unlock(lock_t *l) {
      enableInterrupts();
}
```

Disadvantages?

A useful idea for small critical sections, especially within OS … but not ideal for user-space

Don't want to allow user-space code disable interrupts

What if process crashes or runs away with the processor without yielding?

Stopping interrupts for too long is bad (can lose data)

Only works on uniprocessors

Review: Locks with Atomics

Use hardware support for atomic operations

Atomic operation: No other instructions can interleave

Examples of atomic operations

- Loads and stores of words (aligned, hardware-native load/store size)
  - Load r1, B
  - Store r1, A
- **Special hardware instructions**
  - **Test&Set**
  - **Compare&Swap**
  - **Fetch&Add**

Review: LOCK with atomic XCHG

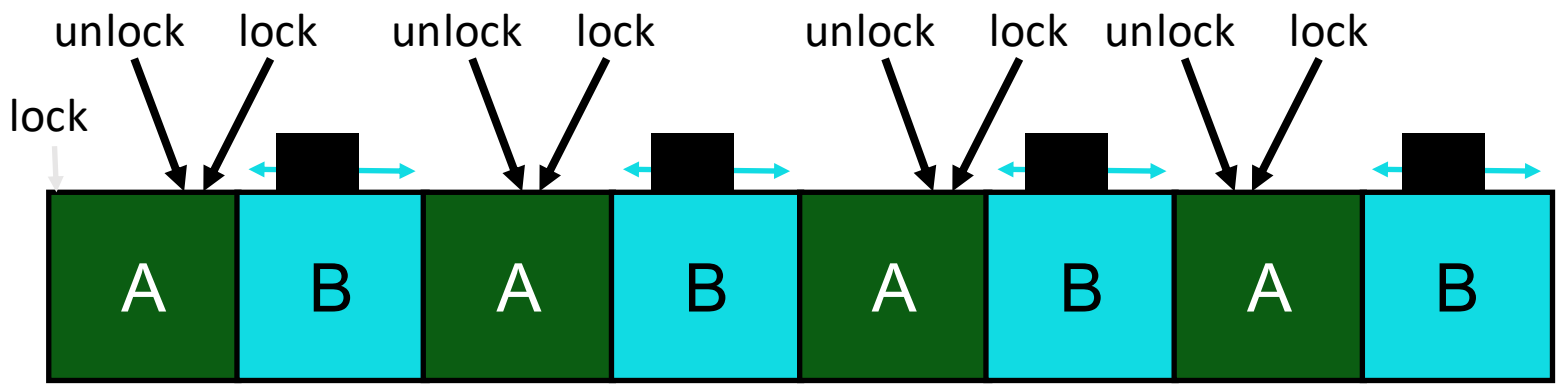
```
typedef struct __lock_t {
      int flag;
} lock_t;

void init(lock_t *lock) {
      lock->flag = 0;
}

void acquire(lock_t *lock) {
      while(xchg(&lock->flag, 1) == 1) ;
      // spin-wait (do nothing)
}

void release(lock_t *lock) {
      lock->flag = 0; // no atomics here
}
```

Review: Spinlocks can starve; unfair



OS Scheduler is typically independent of locks/unlocks

Review: A fair lock with atomic FAA

```
typedef struct __lock_t {
        int ticket;
        int turn;
}

void lock_init(lock_t *lock) {
        lock->ticket = 0;
        lock->turn = 0;
}
```

```
void lock(lock_t *lock) {
        int myturn = FAA(&lock->ticket);
        while (lock->turn != myturn); // spin
}

void unlock (lock_t *lock) {
        FAA(&lock->turn);
}
```

Review: Spinlock Performance

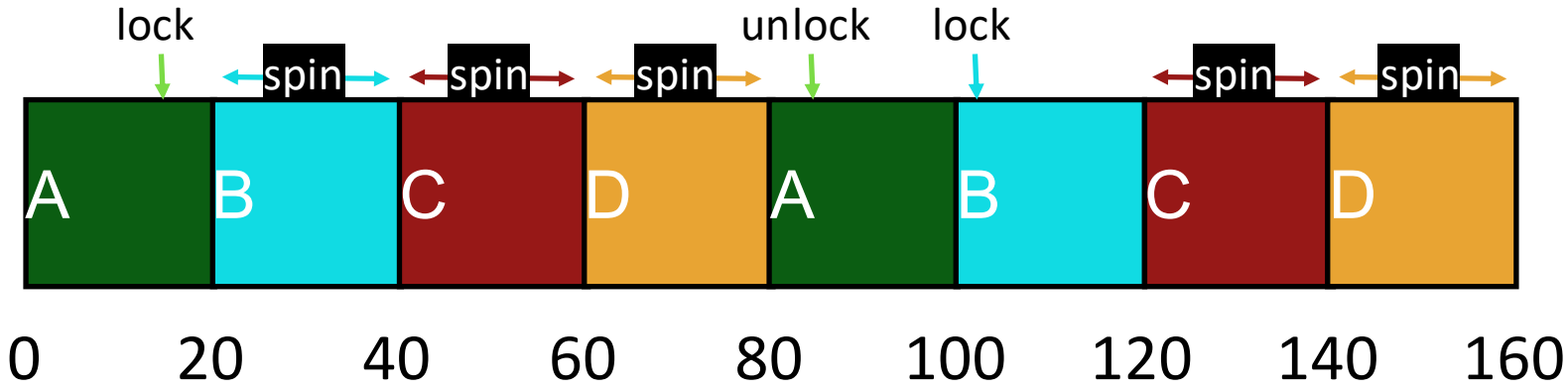
Fast when…

- many CPUs
- locks held a short time
- advantage: avoid context switch

Slow when…

- one CPU
- locks held a long time
- disadvantage: spinning is wasteful

Spinlocks could be CPU-inefficient



CPU scheduler may run **B** instead of **A**
even though **B** is waiting for **A**

Significant inefficiency on uniprocessors
(e.g. N-1 threads waiting one quantum each for N'th thread to release the lock)

Could we just `yield()` when waiting?

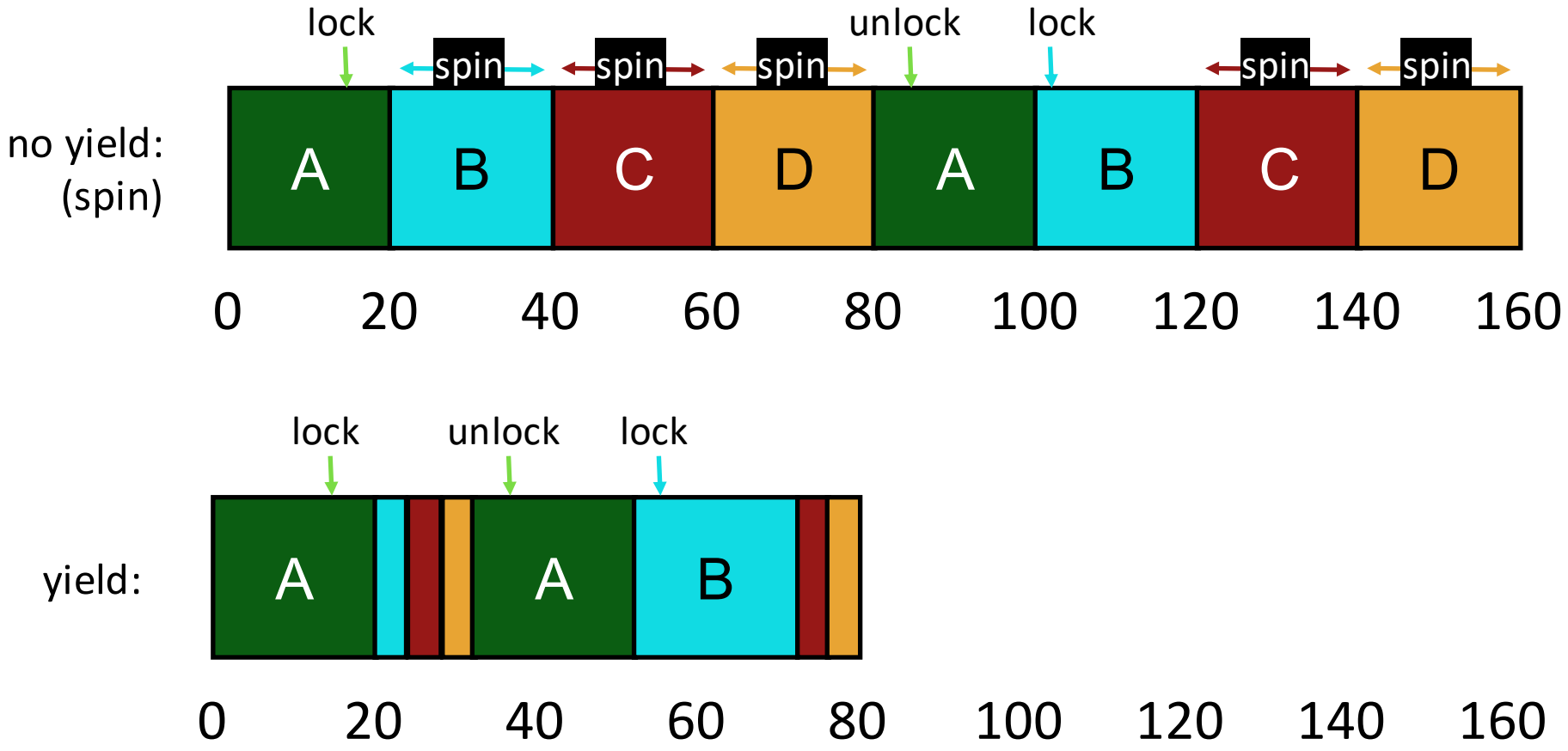
```
typedef struct __lock_t {
        int ticket;
        int turn;
}

void lock_init(lock_t *lock) {
        lock->ticket = 0;
        lock->turn = 0;
}
```

```
void lock(lock_t *lock) {
        int myturn = FAA(&lock->ticket);
        while(lock->turn != myturn)
                yield();
}

void unlock(lock_t *lock) {
        FAA(&lock->turn);
}
```

`Yield` Instead of Spin



Spinlock vs. `yield` Performance

How much CPU wasted?

Without yield: O(threads * **time_slice**)
With yield: O(threads * **context_switch**)

Even with yield, spinning is slow with high thread contention

Can we block (don't schedule a thread until unlocked) and put the thread on a waiting queue instead of check and yield each time?

OS support in implementing locks is helpful!

Also useful to enforce fairness (e.g., unblock in order of lock())

Sleep, not spin

```
typedef struct __lock_t {
    int flag;
    int guard;
    queue_t *q;
} lock_t;

void lock_init(lock_t *m) {
    m->flag  = 0;
    m->guard = 0;
    queue_init(m->q);
}
```

`flag` is the lock

`guard` protects Q

```
void lock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; //acquire guard lock by spinning
    if (m->flag == 0) {
        m->flag = 1; // lock is acquired
        m->guard = 0;
    } else {
        queue_add(m->q, gettid());
        m->guard = 0;
        park();
    }
}
```

Solaris: `park()` asks the OS to put the thread to sleep. OS can reference the thread (later) from the lock-specific queue `m->q`

Sleep, not spin

```
typedef struct __lock_t {
    int flag;
    int guard;
    queue_t *q;
} lock_t;

void lock_init(lock_t *m) {
    m->flag  = 0;
    m->guard = 0;
    queue_init(m->q);
}
```

`flag` is the lock

`guard` protects Q

```
void lock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; //acquire guard lock by spinning
    if (m->flag == 0) {
        m->flag = 1; // lock is acquired
        m->guard = 0;
    } else {
        queue_add(m->q, gettid());
        m->guard = 0;
        park();
    }
}
```

`unpark(tid)` wakes up a previously `park()`-ed thread `tid`

```
void unlock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; //acquire guard lock by spinning
    if (queue_empty(m->q))
        m->flag = 0; // let go of lock; no one wants it
    else
        unpark(queue_remove(m->q)); // hold lock
                                    // (for next thread!)
    m->guard = 0;
}
```

Two questions

- Is spinning ok?
- Do you see any race conditions?

```
void lock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; //acquire guard lock by spinning
    if (m->flag == 0) {
        m->flag = 1; // lock is acquired
        m->guard = 0;
    } else {
        queue_add(m->q, gettid());
        m->guard = 0;
        park();
    }
}
```

wakeup/wait race

```
void unlock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; //acquire guard lock by spinning
    if (queue_empty(m->q))
        m->flag = 0; // let go of lock; no one wants it
    else
        unpark(queue_remove(m->q)); // hold lock
                                    // (for next thread!)
    m->guard = 0;
}
```

Concurrency Objectives

Mutual exclusion (e.g., A and B don't run at same time)

- solved with *locks*
- A can run before or after B

Ordering (e.g., B runs *only after* A does something)

- solved with condition variables

Counting (e.g., only k resources can be granted)

- Solved with semaphores

Can you implement ordering with locks?
If not, why not?